

Performance Impact of Using MQ Selectors in IBM MQ V9.4.4

Author: Aneesha Yerrathota

Introduction

IBM MQ selectors provide a mechanism for message consumers to filter messages based on properties. This feature is highly useful in complex application designs where consumers need to process subsets of messages from a queue based on property values. While selectors increase flexibility and reduce application-side filtering logic, they can also introduce processing overhead within the queue manager, affecting overall throughput and CPU utilization. This paper examines the performance impact of selectors and the underlying reasons behind it.

Test Environment

Component	Version/Details
Platform	Linux x86-64
Tools	MQ-CPH
IBM MQ Version	V9.4.4
Network Bandwidth	100 Gb/s
Application Connection	Client Binding Mode
Message Size	2KB non-persistent
Test Type	Requester/Responder

Hardware

QM Host & Client Hosts:

- ThinkSystem SR630 V2
- 2 x 16 core CPUs
Intel(R) Xeon(R) Gold 6346 CPU @ 3.10GHz
- 256GB RAM
- 100Gb network card

Test Scenarios

To evaluate the performance implications of using selectors, a structured set of requester-responder scenarios was designed. All tests were run using client bindings with a message length of 2 KB (non-persistent), unless otherwise specified. The objective was to compare message retrieval efficiency when using correlation IDs, no selectors, and custom selectors — across both single-queue and multi-queue configurations, as well as under deep queue scenario.

1. Baseline

In the baseline setup, the requester and responder applications exchange messages using standard correlation-ID matching without any message selectors enabled. This represents typical MQ request-reply behaviour and serves as the reference for comparison. The detailed description of the request-reply test scenario is available [here](#) (see *Section 3.1 of the MQ V9.4 Performance Report for xLinux*).

2. Selectors Enabled

This scenario was identical to the baseline except that message selectors were enabled on the consumer side. Messages carried the same identifying properties, but the responder retrieved them using a selector expression rather than direct correlation matching. This configuration helps determine the impact of selector parsing and property filtering.

3. Custom Selector Evaluation

In this configuration, the responder applied custom selector expressions that filtered messages based on MQMD fields such as `Root.MQMD.MsgId`, `Root.MQMD.CorrelId`, `Root.MQMD.MsgType`, or `Root.MQMD.Priority`.

4. Property-Based Filtering Using Message Handles

In this configuration, each request message carries a custom `CorrelId` stored as an RFH2 property is carried forward using message handles and propagated during message reuse. The selector then filters on the `Root.MQMD.CorrelId` to retrieve the correct message. This test shows overhead caused by using user-defined value in RFH2 header and filtering based on this custom generated `correlId`.

5. Deep Queue Retrieval

To study how queue depth and message position affect selector performance, additional tests were conducted where queues were pre-populated with a large number of messages. The objective was to measure the time and throughput required to retrieve specific messages from deep queues under different configurations.

Test Methodology:

Each test scenario followed a two-phase approach:

Load Phase: Messages with specific IDs (ID1 and ID2) were loaded into the queue using sender operations

Unload Phase: Messages were retrieved in two different orders to measure positional impact:

Case A: Retrieve ID1 messages first, then ID2 messages

Case B: Retrieve ID2 messages first, then ID1 messages

This approach helps identify whether deep queue and message position in the queue (head vs. tail) affects retrieval performance when using different selector configurations.

CPH Configuration Summary for Test Scenarios

Test Scenario	Queue Configuration	-cs	-co	-gs	-mh
Baseline	Single Queue	No	Yes	No	No
Baseline	Multi Queue	No	Yes	No	No
Selectors Enabled	Single Queue	Yes	Yes	No	No
Selectors Enabled	Multi Queue	Yes	Yes	No	No
Custom Selector (MQMD fields)	Single Queue	Yes	Yes	Yes	No
Custom Selector (MQMD fields)	Multi Queue	Yes	Yes	Yes	No
Property-Based Filtering (RFH2)	Single Queue	Yes	Yes	No	Yes
Property-Based Filtering (RFH2)	Multi Queue	Yes	Yes	No	Yes
Deep Queue Retrieval	Single Queue	Yes	Varies	No	Varies

Observations

This section summarises the performance behaviour observed across the different scenarios. Measurements focused on throughput (roundtrips/second) and CPU utilisation on the queue manager.

Each round trip in a requester/responder scenario is comprised of 2 messages (a request message and a reply message) so 100 round-trips/sec (for example) equals 200 round-trips/sec

Baseline

The baseline tests use IBM MQ's native Correlation-ID matching (MQGMO_MATCH_CORREL_ID) without any selectors. This mechanism retrieves replies by directly matching the CorrelId in the MQMD, which requires minimal processing inside the queue manager. As a result, this configuration provides the most efficient message retrieval path in MQ.

In the Multi-Queue setup, throughput remains consistently high at around 319k roundtrips/sec as client load increases. In contrast, the Single-Queue setup peaks near 153k roundtrips/sec and then gradually declines to 139k roundtrips/sec at higher number of clients. This behaviour corresponds directly with the single-queue bottleneck, where multiple responders interact with the same queue.

Queue manager (server) and client CPU usage follow the same pattern:

- Multi-Queue CPU rises steadily in line with its stable throughput.
- Single-Queue CPU decreases at higher number of clients as overall throughput declines.

Overall, the baseline establishes the maximum efficiency achievable using Correlation-ID matching without selectors and serves as the reference point for evaluating all subsequent selector-based tests.

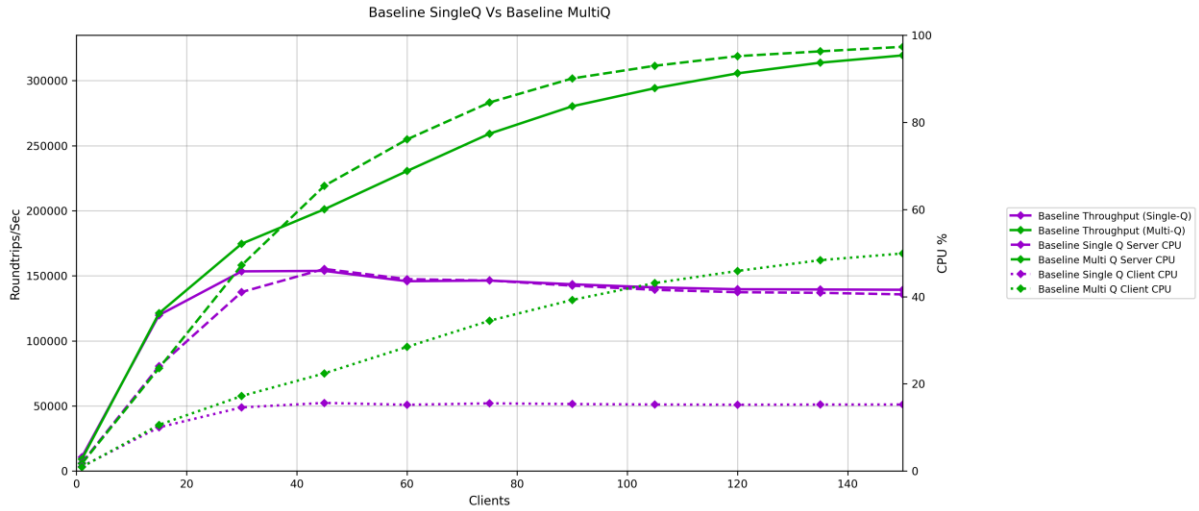


Figure 1- 2KB Baseline Single Queue Vs Baseline Multi Queue

Selectors Enabled

Single Queue :

Enabling selectors on a single queue causes a steep drop in throughput, falling by approximately 87-93% compared to the baseline at moderate to high client loads. CPU utilization is also much lower, not because the selector path is more efficient, but because the system processes far fewer messages overall, demonstrating that selector-based retrieval adds significant overhead and sharply limits scalability compared to direct correlation-ID matching.

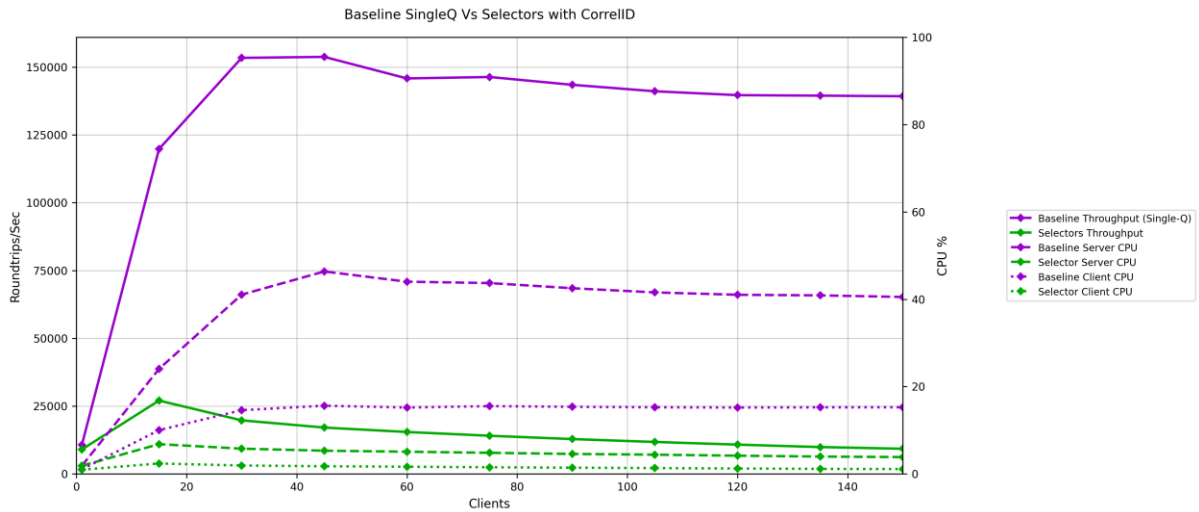


Figure 2- 2KB Baseline Single Queue Vs Selectors Enabled Single Queue

Multi Queue:

In the multi-queue setup, selector processing causes throughput reductions ranging from 29% at moderate loads to 88% at high loads compared to the baseline. While the system performs far better than the single-queue selector case due to load distribution, selector evaluation still imposes significant overhead that limits its scalability well below the baseline, particularly as client count increases.

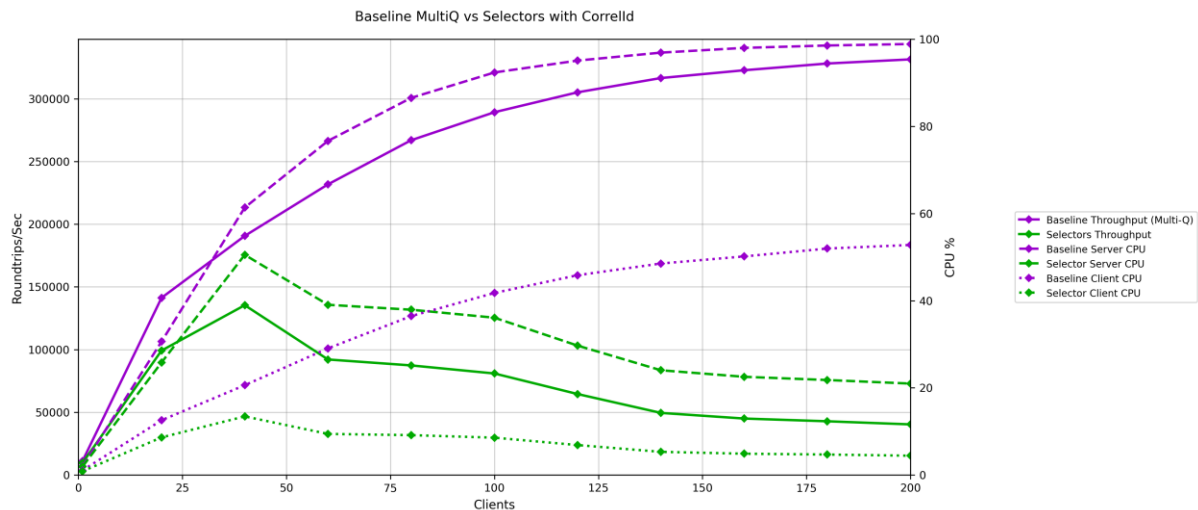


Figure 3- 2KB Baseline Multi Queue Vs Selectors Enabled Multi Queue

Custom Selector Evaluation (MQMD Header Fields)

Single Queue:

This test uses the selector expression “Root.MQMD.MsgId IS NOT NULL “to illustrate how custom selector evaluation affects performance on a single queue. Applying a selector changes how messages are retrieved, adding work to the get path and limiting how much load the system can sustain compared to the baseline, which continues to scale efficiently using direct MQMD-based matching.

Because the selector-enabled test cannot drive the same message rate, overall CPU usage remains lower throughout the run. The baseline, unaffected by this overhead, maintains higher throughput and correspondingly higher CPU utilization as the number of clients increases.

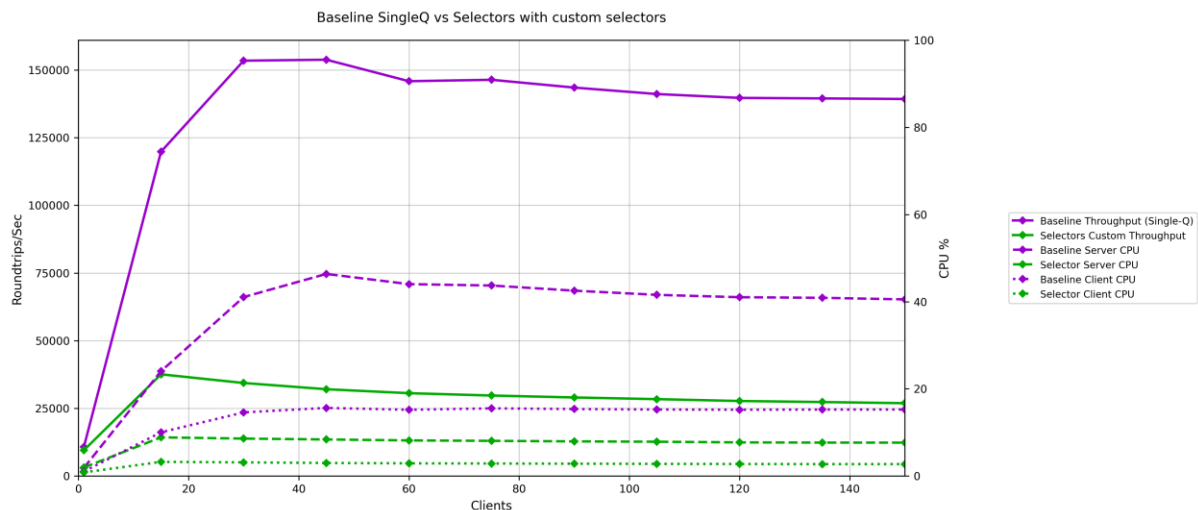


Figure 4- 2KB Baseline Single Queue Vs Custom Selectors Enabled Single Queue

Multi Queue:

Using the custom selector in a multi-queue setup improves throughput compared to the single-queue selector case because the load is spread across multiple queues, but it still does not approach baseline performance. The selector overhead remains the limiting factor, keeping both throughput and CPU utilization below what the system achieves without selectors. Even with additional queues, the baseline continues to scale more effectively.

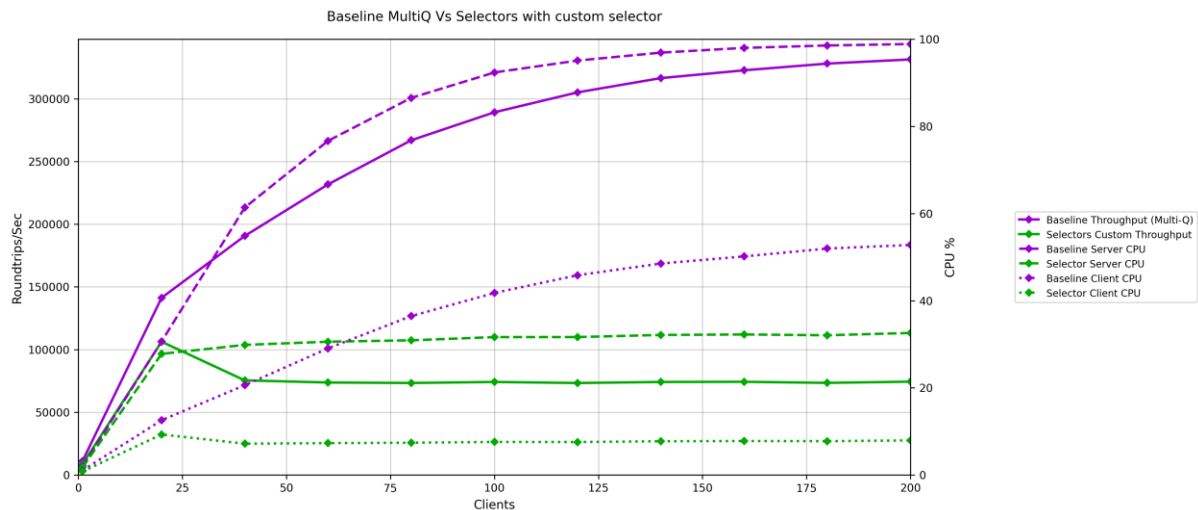


Figure 5- 2KB Baseline Multi Queue Vs Custom Selectors Enabled Multi Queue

Property-Based Filtering Using Message Handles

Single Queue:

Using selectors in combination with a custom CorrelId carried through an RFH2 property introduces extra processing overhead on the get path. Because selector evaluation is involved, the queue manager cannot take advantage of the efficient MQMD-based match used in the baseline. As a result, throughput drops significantly as client load increases, and CPU usage remains lower simply because the system is unable to drive the same volume of messages.

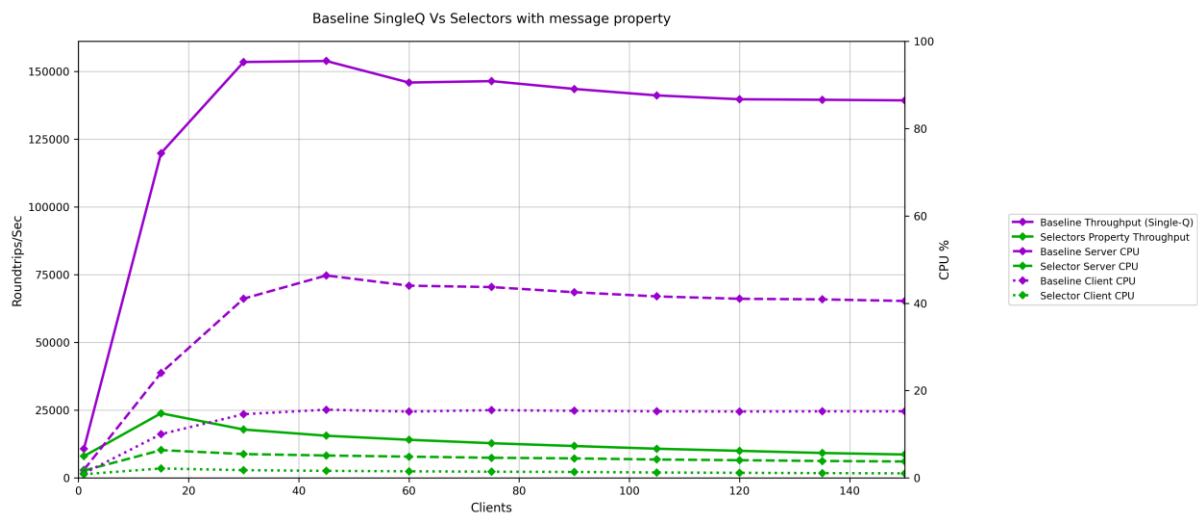


Figure 6- 2KB Baseline Single Queue Vs Custom Property Selectors Enabled Single Queue

Multi Queue:

Using the selector enabled retrieval in the multi-queue test results in noticeably lower throughput than the baseline, although the system performs better than the single-queue selector run due to the additional parallelism available. CPU usage follows the same pattern—reduced values in the selector test simply reflect the lower message flow rather than improved efficiency. Overall, multi-queue processing helps reduce the impact of custom generated correlId filtering, but the baseline still delivers additional performance across all clients.

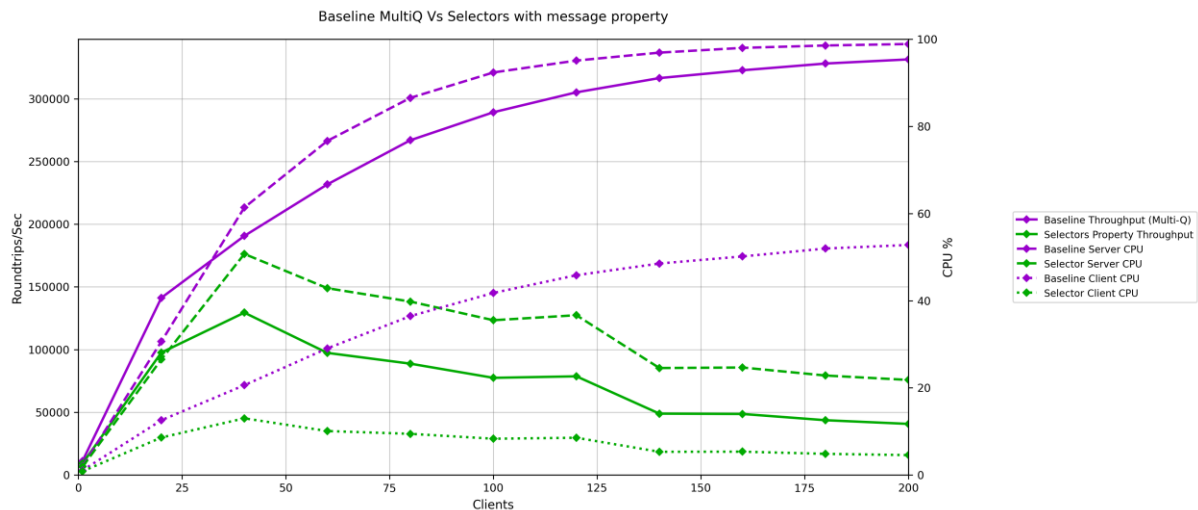


Figure 7- 2KB Baseline Multi Queue Vs Custom Property Selectors Enabled Multi Queue

Deep Queue Retrieval

To study how queue depth and message position affect selector performance, tests were conducted where queues were pre-populated with 2 million messages (1 million each for ID1 and ID2). To ensure statistical reliability, each deep queue scenario was executed four times with a 5-second interval between iterations

Deep Queue Baseline Test(CPH parameter -co flag is set to false):

In the baseline configuration, messages were retrieved without using selectors or custom correlation handling. This represents the standard MQ message retrieval behaviour and serves as the reference point for comparing selector-based deep queue performance.

When retrieving 1 million messages from the queue head first (Case A, ID1), average throughput reached 24,704 roundtrips/sec—approximately 5% higher than when the same messages were retrieved after tail messages had been removed (Case B, ID1: 23,553 roundtrips/sec). Tail message retrieval showed minimal variation between cases (23,715-23,797 roundtrips/sec), indicating that position has a minimal impact on message retrieval.

Baseline Summary

-----Case A (Unload ID1→ID2)-----

	Average Rate(messages/sec)	Time(seconds)
ID1	24703.8	40.73
ID2	23797.17	42.2

-----Case B (Unload ID2→ID1)-----

ID1	23553.3	42.63
ID2	23715.15	42.39

Deep Queue with CorrelationId Optimization(CPH parameter -co is set to true):

This test enabled correlationId optimization to evaluate MQ's correlationId-based retrieval performance from deep queues. There is a minimal overhead seen due to correlationId matching.

Deep Queue with CorrelationId Summary

-----Case A (Unload ID1→ID2)-----

	Average Rate(messages /sec)	Time(seconds)
ID1	22239.6	44.96
ID2	23326.6	43.05

-----Case B (Unload ID2→ID1)-----

ID1	23736.35	42.28
ID2	22892.7	43.8

Deep Queue with Selectors(CPH Parameter -cs is set to true):

In this configuration, message selectors were enabled for retrieving messages from the deep queue. This tests the impact of selector evaluation when scanning through a large number of messages to find matching entries. Here we can see visible drop in the throughput and increase in the processing time.

Deep Queue with Selectors Summary

-----Case A (Unload ID1→ID2)-----

	Average Rate(messages /sec)	Time(seconds)
ID1	21539.03	46.7
ID2	20521.08	48.92

-----Case B (Unload ID2→ID1)-----

ID1	21573.15	46.63
ID2	19496.58	51.51

Deep Queue with Message Handles(CPH parameters -cs and -mh are set to true):

This test combines selector-based retrieval with message handle usage for property management. Messages carry custom properties that are propagated using message handles, and selectors filter based on these properties when retrieving from the deep queue. This test results confirms the overhead caused due to selectors evaluation and processing of message handles.

Deep Queue with Message Handles Summary

-----Case A (Unload ID1→ID2)-----

	Average Rate(messages /sec)	Time(seconds)
ID1	20585.53	48.8
ID2	17629.12	56.93

-----Case B (Unload ID2→ID1)-----

ID1	19265.38	52.11
ID2	16641.5	60.09

- The correlation optimization scenario delivers 3.7% lower average throughput compared to baseline, with the most significant impact on initial head-message (ID1) retrieval (10% slower).
- Selector-based retrieval shows 13.2% average throughput reduction, with the greatest impact on tail message retrieval (17.8% slower in Case B)
- Message handle configuration exhibits the most severe performance degradation at 22.6% lower throughput, with tail message (ID2) retrieval suffering up to 29.8% reduction.
- The combined overhead of selector evaluation, message property handling, and custom correlation-ID propagation through RFH2 headers creates substantial processing burden with deep queue scenarios.

Conclusion

These test results can be used as a reference when making design decisions for production environments. Message selectors should be applied based on specific production requirements, with an understanding of the associated performance trade-offs. For workloads requiring high throughput and low latency, MQ's native correlation-ID matching remains the more efficient approach